Dictionary

→ A dictionary is a dynamic set ADT.

→ ADT is an object with a generic discription, independent of implementation details.

→ A dictionary is a container of elements, where each element is a pair of key and value and every value is associated with the corresponding keys.

→ It is also a type of DS.

Basic operations of dictionary ADT.

1. Insert (X, D): It insert element (x) in dictionary (D)
2. Delete (X, D): Delete (X) from dictionary (D)
3. Search (X, D): Search element (X) in (D)
4. Member (X, D): Gives true if (X) belong to (D)
5. Size (D): Returns the size of dictionary.
6. Max (D): Returns the maximum/ biggest element of (D)

Ways of implementation.

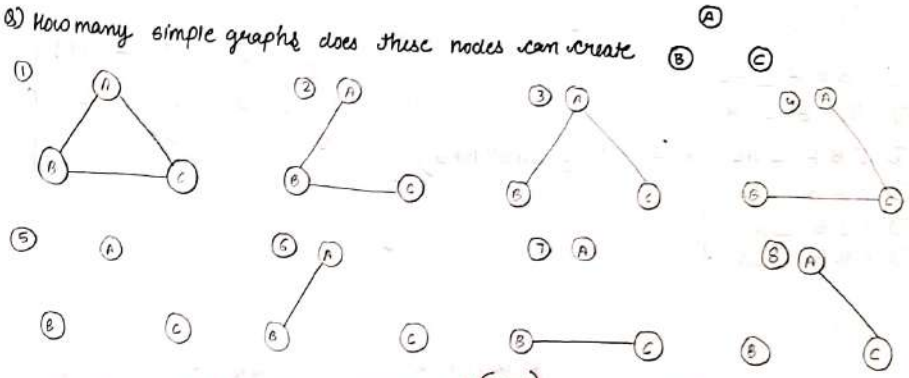
1) Fixed length array
2) Sorted linked list.
3) Hashing
4) Trees (BST, Balanced BST, Splay, M-way, Tries)

Graphs

Simple graph properties:

→ Undirected
→ Unweighted
→ No self loops
→ No parallel edges.

Q) How many simple graphs does these nodes can create



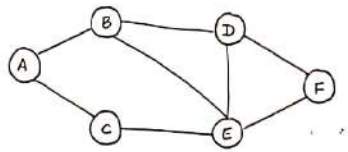
no. of max^m possible edges: $\frac{n(n-1)}{2}$ ∴ n → nodes

no. of simple graphs: 2^n ∴ n → edges

BFS	DFS
BREADTH FIRST SEARCH.	→ DEPTH FIRST SEARCH
→ In BFS nodes are traversed level by level.	→ In DFS nodes are traversed depth wise.
→ BFS uses QUEUE data str.	→ DFS uses STACK data str.
→ BFS is slower as memory consumption is high.	→ DFS is faster as it requires less memory.
→ Does not support backtracking.	→ Supports backtracking.
→ It is suitable for finding the shortest path.	→ Suitable for topological sorting & finding the strongly connected componen
→ Time complexity O(|V|+|E|).	→ Time complexity O(|V| +|E|).

Scanned with OKEN Scanner

Q) Find total no. of BFS possible if (E) is source node.



E $\underbrace{CBDE}_{4!\ \text{comb}^n}$ AC
CA → (24 BFS)

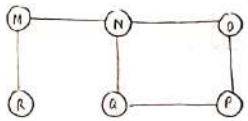
(ii) Taking (D) as source node

D B E F _ AC
D B F E _ CA
D E B F _ AC
D E F B _ CA
D F E B _ AC
D F B E _ CA

3! → (6 comb^n BFS)

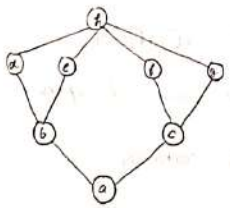
no. of BFS comb^n = $n!$
n: no. of adjacent node of source node

Q) Which is valid BFS comb^n:



a) MNOPQR
b) NQMPOR
c) QMNROP
d) PQRNMR

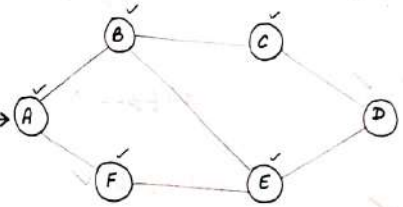
Q) Which is valid BFS comb^n:



a) abcdefgh
b) acbfgedh
c) abcdegh
d) acbgedh

BFS (u)

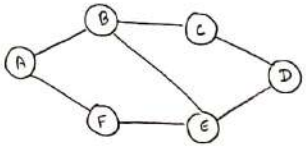
{
mark (u) as visited &
enqueue (u)
while (queue is not empty) {
X = dequeue (u), Traverse (X)
put all unvisited nodes of u in queue &
mark them visited
}
}



① Let (A) be source node : visited = A	A						Traverse: A
② Dequeue (A) : visited = ABF	~~A~~	B	F				Traverse: A
③ Dequeue (B) : visited = ABFCE	~~A~~	~~B~~	F	C	E		Traverse: AB
④ Dequeue (F) : visited = ABFCE	~~A~~	~~B~~	~~F~~	C	E		Traverse: ABF
⑤ Dequeue (C) : visited = ABFCED	~~A~~	~~B~~	~~F~~	~~C~~	E	D	Traverse: ABFC
⑥ Dequeue (E) : visited = ABFCED	~~A~~	~~B~~	~~F~~	~~C~~	~~E~~	D	Traverse: ABFCE
⑦ Dequeue (D) : visited = ABFCED	~~A~~	~~B~~	~~F~~	~~C~~	~~E~~	~~D~~	Traverse: ABFCED

Scanned with OKEN Scanner

DFS

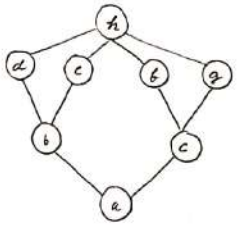


RD (6): A → B → C → D → E → F

RD (5): A → B → E → F, D → C

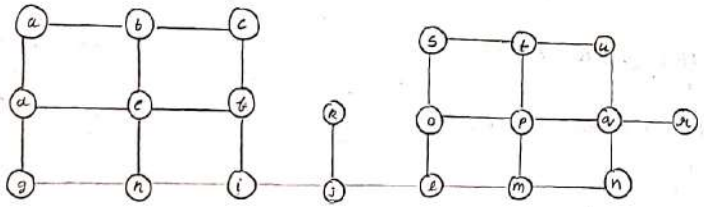
RD (5): A → B → E → F, D → C

Q) Identify the valid DFS.



1) abehfgdc ✗
2) acfhgebd ✓
3) abehdfeg ✓
4) acfhebdg ✓
5) abcfhdeg ✗
6) abdhfgce ✗
7) abdhgcfe ✓
8) acbdhefg ✗
9) acfhgdeb ✗
10) acghebdf ✓
11) acbehfgd ✗

Q) Find the max^m possible Recursion Depth for following graph:



a → b → c → f → e → d → g → h → i → j → l → m → n → q → p → o → s → t → u

j → k, q → r

max: 19

a → d → g

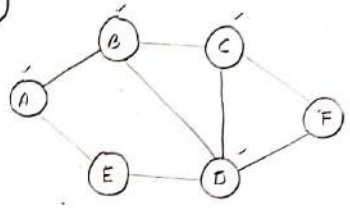
i → h → e → b → c → f

j → k

l → m → p → t → s → o

t → u → q → h → r

Q)



Traversing: A B C D E F

Processing: A B C D E D D F C B A

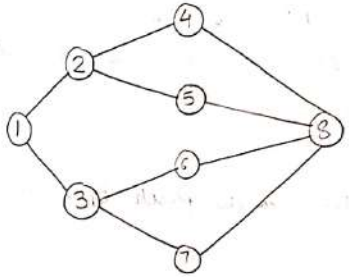
i) DFS Traversal: ABCDEF

ii) element not pushed in stack: E, F

iii) elemenent pushed more than once: D

Q) Perform DFS in given graph with source as vertex (1).

Vertex	adjacent
1	2, 3
2	1, 4, 5
3	1, 6, 7
4	2, 8
5	2, 8
6	3, 8
7	3, 8
8	4, 5, 6, 7



Traversal: 1 2 4 8 5 6 3 7

processing: 1 2 4 8 5 8 6 3 7

i) DFS traversal: 1 2 4 8 5 6 3 7

ii) element not pushed in stack: 5

iii) Element pushed more than once: 8

Scanned with OKEN Scanner

$\boxed{DFS(u)}$

```
{
  Mark u as visited
  while ( there is a neighbour 'V' of 'u' which is unvisited ){
        DFS(V)
  }
}
```

Edges in DFS

① **Tree edge:** edges which are part of DFS traversal.

② **Forward edge:** $e_i(x,y)$

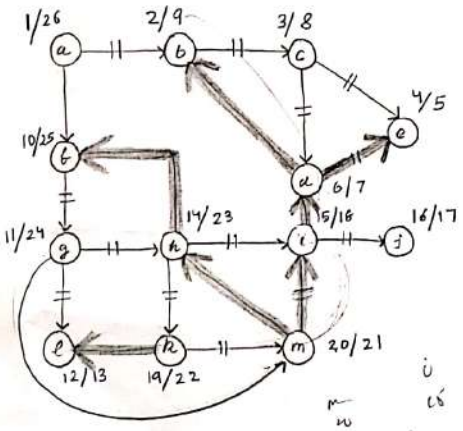
→ y appears after x, and there is a path from $x \to y$.

③ **Back edge:** $e_i(x,y)$

→ y appears before x & there is a path from y to x.

④ **Cross edge:** $e_i(x,y)$

→ There is no path b/w x & y.



Tree edge: {(a,b) (b,c) (c,d) (c,e), (a,f) (f,g) (g,l) (g,h), (h,i) (i,j) (h,k) (k,m)}

g m
|| 20
x → y forward

① d → b (Back edge)
x y
6 2
x y

② d → e (cross edge)
x y
6 4

③ h → f (back edge)
x y
14 10

④ i → d (cross edge)
x y
15 6

⑤ k → l (cross edge)
x y
19 12

⑥ m → h (Back edge)
x y
20 14

⑦ m → i (cross edge)
x y
20 15

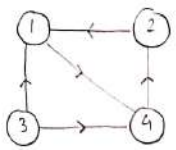
⑧ g → m (forward edge)
x y
11 20

Graph Representation:

① **ADJACENCY MATRIX:**

→ Adjacency matrix is an $(n \times n)$ matrix where (n) no. of vertices and value $A_{ij} = [1]$ if i and j are connected otherwise $[0]$.

→ The biggest disadvantage of adjacency matrix is $O(n^2)$ complexity and time complexity begin one of the biggest advantage for searching, insertion and deletion take $O(1)$ time.

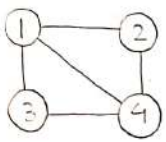


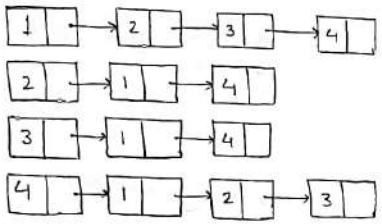
	1	2	3	4
1	0	0	0	1
2	1	0	0	0
3	1	0	0	1
4	0	1	0	0

Scanned with OKEN Scanner

② ADJACENCY LIST :

→ In adjacency list both the edges and vertices are represented using linked list.

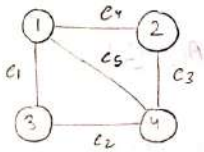




③ INCIDENCE MATRIX :

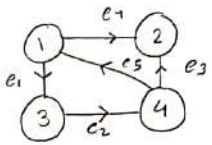
→ There should be no self loop and parallel edges, and it is $(n \times e)$ matrix

→ (n) Vertices represent rows & (e) edges represent columns.



	e_1	e_2	e_3	e_4	e_5
1	1	0	0	1	1
2	0	0	1	1	0
3	1	1	0	0	0
4	0	1	1	0	1

Sum of column of undirected graph will always be exactly (2)



incoming : -1
outgoing : +1
no edge : 0

	e_1	e_2	e_3	e_4	e_5
1	+1	0	0	+1	-1
2	0	0	-1	-1	0
3	-1	+1	0	0	0
4	0	-1	+1	0	+1

Sum of column of directed graph will always be (0)

Topological Sorting / Ordering

→ It is a linear sorting / ordering of its vertices such that for every directed edge or vertex $(u \rightarrow v)$, (u) come before vertex (v) in the ordering.

→ Graph should be directed acyclic graph, which means there should not be any cycle in graph.

→ There are 2 types of implementation :

(i) BFS Implementation [KAHN'S ALGO]

(ii) DFS Implementation [KOSARAJU ALGO]

→ Many real world appl^n can be modeled as a graph with directed edges, where some events must occur before others.

Eg :
- Semester - class prerequisite.
- Program dependencies
- Event scheduling
- Assembly instructions
- Program - built dependencies

→ The idea is to first build programs, without dependency and then move from there to the next level.

→ Time comp. is linear $O(V+E)$, topological ordering are not unique & all the trees have a topological ordering since they don't contain a cycle.

Scanned with OKEN Scanner

KAHN'S Algo

1) Compute the indegree of every node and put the node in a queue.

2) Remove the node from queue then add it to the answer list / array.

3) Decrease indegree by (1) for all its neighbouring nodes, if the indegree of neighboring nodes is reduced to (0) then add it to the queue.

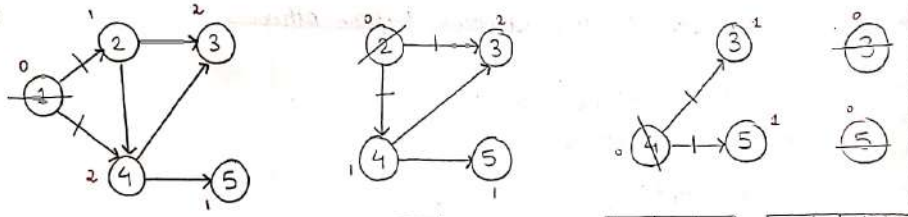
4) Repeat steps 2 & 3 until the queue is empty.

5) Return the answer / array / list.

→ Space complexity: $O(V)$

→ Time complexity: $O(V+E)$

8)



1

1	2

1	2	4

1	2	4	3	5

KOSARAJU ALGO

→ It's a 2 pass DFS algo.

<u>1st pass</u> : used to find the topological sorting, 1st and 2nd pass combined helps to find strongly connected component.

① Create a stack to store the nodes.

② Initialize the visited array of (n) size, to keep record of visited nodes.

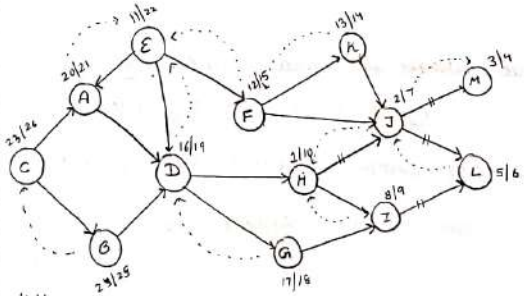
③ Run a node from (0 → n), if node is not mark (True) in visited array, call the recursive f^n for the topological sort and perform the following steps.

[3.1] Mark the current node as True in visited ar.

[3.2] Run a loop on all the nodes which have a directed edge to the current node, if the node is not marked true in the visited array, then recursively call the topological sort f^n on the node

[3.3] Push the current node in stack.

④ Now Pop all the elements from stack and push it to the result array & return the result.

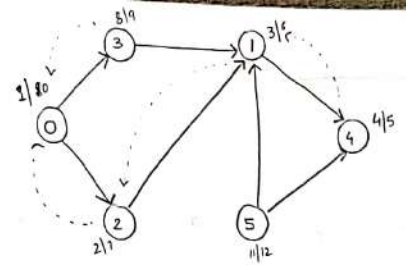


i) H
ii) E
iii) C

TO: C B E A D G F K H I J L M

C
B
E
A
D
G
F
K
H
I
J
L
M

Q)



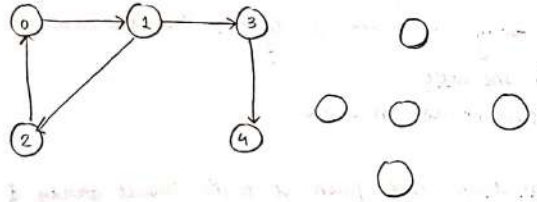
5
0
3
2
1
4

TO: 5 0 3 2 1 4

STRONGLY CONNECTED COMPONENT :

→ For undirected graph we have concept of connected components & for directed graph we have the concept of strongly connected components.

→ Component of a graph in which every vertex has a path to every other vertex in that graph is called as connected component.



→ Kosaraju's algo is efficient and works in linear time $O(v+e)$.

→ It's a fundamental algo for analysing a str. of directed graph.

→ It is used in various applications like compiler optimization, software engineering, recommendation system, robotics and path planning and transportation networks.

2nd pass:

① Transpose the graph, which means reverse the edges b/w every vertices.

② Second DFS pass (Identify SCCs):

① Create a new set/array to keep track of visited nodes.

② Initialize the list to store SCCs.

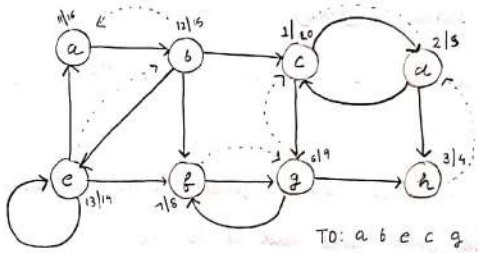
③ While the stack != Empty.

[3.1] Pop a node from the stack.

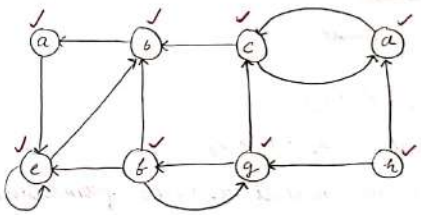
[3.2] If the node has not been visited perform a DFS, Starting from this node in the reversed graph, during the DFS, all the nodes visited these nodes will form an SCC, add the SCC to the list of SCCs.

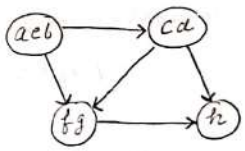
④ Continuity process until stack is empty.

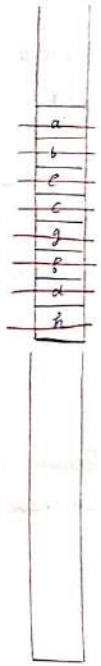
⑤ The list of SCCs now contain the SCC of the actual graph.



TO: a b e c g f d h

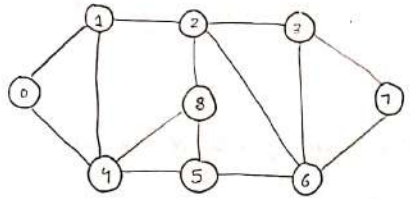






[aeb cd gf h]

DIJKSTRA ALGO



→ This algo is used to find the single source shortest path.

→ It works for directed & undirected graph.

→ It may or may not work for (-ve) weights.

→ If total edge weight of cycle is (-ve) then surely Dijkstra won't work.

→ Space complexity is : $O(V+E)$

→ Time complexity is : $O(V+E\log V)$

- Algorithm steps:

- DIJKSTRA (G, source) {.....

① Create a set (S) to keep track of nodes with known shortest distance

② Create an array to store the current shortest distance estimated from the source, initially all to infinity and source as (zero).

③ Create a priority queue or min heap to maintain the nodes to explore with array values as keys.

④ While (queue is not empty), select the node (U) in queue with smallest array value and remove (U) from queue and add it to set.

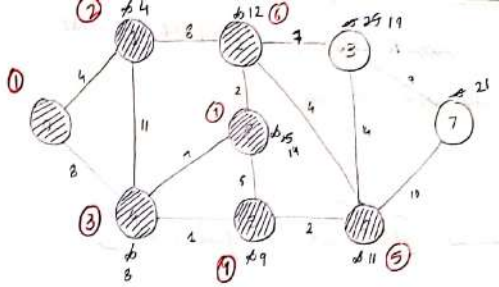
⑤ For each neighbour (V) of (U), first calculate the tentative distance from source to (V) through (U) using this formulae.

$$\text{if } (d(U) + c(U,V) < d(V))$$

$$d(V) = d(U) + c(U,V)$$

⑥ Return the array which now contains the shortest distance from source to node in graph.

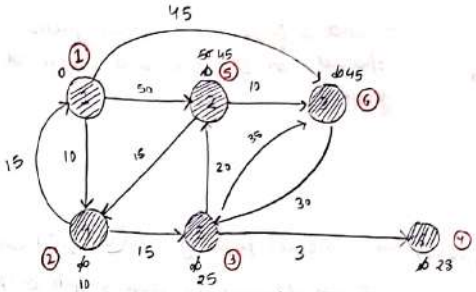
}

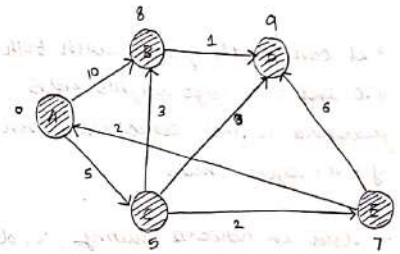


$$\text{if } (d(u) + c(u,v) < d(v))$$

$$d(v) = d(u) - c(u,v)$$

distance cost





Selected vertex	A	B	C	D	E
A	$\boxed{0}$	∞	∞	∞	∞
C		10	$\boxed{5}$	∞	∞
E		8		14	$\boxed{7}$
B		$\boxed{8}$		13	
D				$\boxed{9}$	

Scanned with OKEN Scanner

→ It is used in routing algorithm, network design, finding the shortest path in transportation or communication network.

FLOYD WARSHALL ALGO

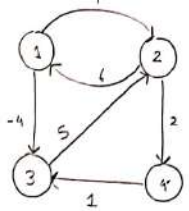
(all pair shortest path)

Dijkstra	Floyd warshall
→ It is used to find the shortest path b/w the specific nodes in a weighted graph, it works well in finding the shortest path from single source to all other node as well.	→ Used to find the shortest path b/w all pair of nodes in a weighted graph.
→ Time complexity: $O(V^2)$ {adjacency matrix} $O(E+V\log V)$ {min heap}	→ Time complexity: $O(V^3)$, It can be less efficient in large graph with many vertices.
→ It can handle graphs with (+ve) weights but may or may not give correct result for (-ve) edge weights.	→ It can handle graphs with both (+ve) and (-ve) edge weights and is guaranteed to find correct result even if (-ve) cycle exists.
→ Used in GPS nav system.	→ Used in network routing, used in analyzing the connectivity in a network.

→ if $(d(u) + c(u,v) < d(v))$

$d(v) = d(u) + c(u,v)$

→ $D^k[i,j] = \text{MIN}\begin{cases} D^{k-1}[i,j] \\ D^{k-1}[i,k] + D^{k-1}[k,j] \end{cases}$



$D^{0 \to k}$

	1	2	3	4
1	0	9	-4	∞
2	6	0	∞	2
3	∞	5	0	∞
4	∞	∞	1	0

$\begin{cases} D^{k-1}[i,j] \\ D^{k-1}[i,k] + D^{k-1}[k,j] \end{cases}$

$D^{1 \leftarrow k}$

	1	2	3	4
1	0	9	-4	∞
2	6	0	2	2
3	∞	5	0	∞
4	∞	∞	1	0

① $D^1[2,3] = \text{MIN}\begin{cases} D^0[2,3], \\ D^0[2,1] + D^0[1,3] \end{cases} \to \text{MIN}\begin{cases} \infty \\ 2 \end{cases} \to ②$

② $D^1[2,4] = \text{MIN}\begin{cases} D^0[2,4], \\ D^0[2,1] + D^0[1,4] \end{cases} = \text{MIN}\begin{cases} 2 \\ \infty \end{cases} \to ②$

③ $D^1[3,2] = \text{MIN}\begin{cases} D^0[3,2], \\ D^0[3,1] + D^0[1,2] \end{cases} = \text{MIN}\begin{cases} 5 \\ \infty \end{cases} \to ⑤$

④ $D^1[3,4] = \text{MIN}\begin{cases} D^0[3,4], \\ D^0[3,1] + D^0[1,4] \end{cases} = \text{MIN}\begin{cases} \infty \\ \infty \end{cases} \to ∞$

⑤ $D^1[4,2] = \text{MIN}\begin{cases} D^0[4,2], \\ D^0[4,1] + D^0[1,2] \end{cases} = \text{MIN}\begin{cases} \infty \\ \infty + 9 \end{cases} \to ∞$

⑥ $D^1[4,3] = \text{MIN}\begin{cases} D^0[4,3] \\ D^0[4,1] + D^0[1,3] \end{cases} = \text{MIN}\begin{cases} 1 \\ \infty + (-4) \end{cases} \to ①$

$D^{2 \to k}$

	1	2	3	4
1	0	9	-4	11
2	6	0	2	2
3	11	5	0	7
4	∞	∞	1	0

① $D^2(1,3) = \text{MIN}\begin{cases} D^1[1,3] \\ D^1[1,2] + D^1[2,3] \end{cases} = \text{MIN}\begin{cases} -4 \\ 9+2 \end{cases} \to (-4)$

② $D^2(1,4) = \text{MIN}\begin{cases} D^1[1,4] \\ D^1[1,2] + D^1[2,4] \end{cases} = \text{MIN}\begin{cases} \infty \\ 9+2 \end{cases} \to (11)$

③ $D^2(3,1) = \text{MIN}\begin{cases} \infty \\ 5+6 \end{cases} \to (11)$ ⑤ $D^2(4,1) = \text{MIN}\begin{cases} \infty \\ \infty + 6 \end{cases} \to ∞$

④ $D^2(3,4) = \text{MIN}\begin{cases} \infty \\ 5+2 \end{cases} \to ①$ ⑥ $D^2(4,3) = \text{MIN}\begin{cases} 1 \\ \infty + 2 \end{cases} \to ①$

$D^{3 \to k}$

	1	2	3	4
1	0	1	-4	3
2	6	0	2	2
3	11	5	0	7
4	12	6	1	0

① $D^3(1,2) = \text{MIN}\begin{cases} 9 \\ -4+5 \end{cases} = ①$

② $D^3(1,4) = \text{MIN}\begin{cases} 11 \\ -4+7 \end{cases} = ③$

③ $D^3(2,1) = \text{MIN}\begin{cases} 6 \\ 2+11 \end{cases} = ⑥$

④ $D^3(2,4) = \text{MIN}\begin{cases} 2 \\ 2+7 \end{cases} = ②$

⑤ $D^3(4,1) = \text{MIN}\begin{cases} \infty \\ 1+11 \end{cases} = ⑫$

⑥ $D^3(4,2) = \text{MIN}\begin{cases} \infty \\ 1+5 \end{cases} = ⑥$

$D^{4 \to k}$

	1	2	3	4
1	0	1	-4	3
2	6	0	2	2
3	11	5	0	7
4	12	6	1	0

① $D^4(1,2) = \text{MIN}\begin{cases} 1 \\ 3+6 \end{cases} \to ①$

② $D^4(1,3)$

Scanned with OKEN Scanner

→ Basic f^n of Floyd Warshal Algo:

```
for (k=1; k<=n; k++){
    for (i=1; i<=n; i++){
        for (j=1; j<=n; j++){
            A[i][j] = Min(A[i][j],
                          A[i][k]+A[k][j]);
        }
    }
}
```

NETWORK FLOW

FORD FULKERSON ALGO:

→ It is used to find the max^m flow to the given network, it iteratively find the augmenting path from SOURCE to the SINK and increases the flow along those paths until nomore augmenting path exists

—— Basic term nology used in network flow:

1) SOURCE → It has all outward edge and no inward edge.
2) SINK → It will have all inward edge and no outward edge.
3) BOTTLENECK CAPACITY → It's the min^m capacity of any edge on the path.
4) RESIDUAL CAPACITY → Original capacity — Current flow.
5) RESIDUAL GRAPH → It is a representation of original network flow graph, but it's modified to reflect the capacity that remains available in each edge after initial flows have been sent.
   → It helps to determine which path can be further augmented.
6) AUGMENTING PATH → It's a path from source → sink in the residual graph.

—— Complexity:

→ Time complexity can vary based on the method used to find the augmented path.

→ Edmund Karp variant which use BFS to find augmented path has TC of $O(VE^2)$.

→ Ford Fulkerson Algo is used in app^n such as network routing, transportation network, network design & flow of goods & supply chain.

Scanned with OKEN Scanner

Algorithm :

fordfulkerson (graph, source, sink) {

1) Initialize flow = 0.

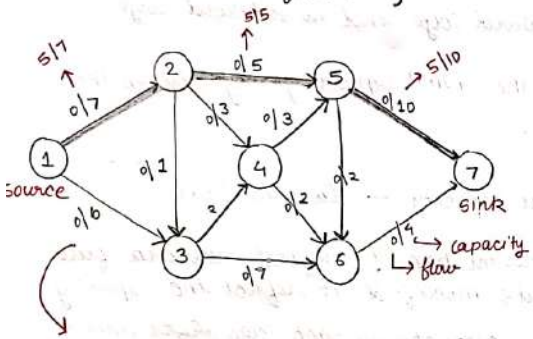
2) While (there exists an augmenting path (P) in residual graph.)

[2.1] Find the bottleneck capacity.

[2.2] Augment the flow around path (P), by bottle neck capacity.

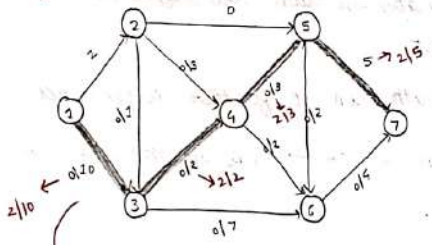
[2.3] Update the residual graph.

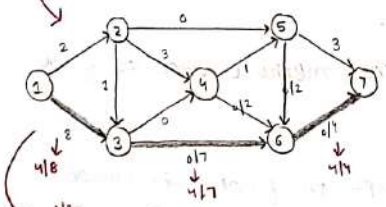
3) Return Max^m flow. }

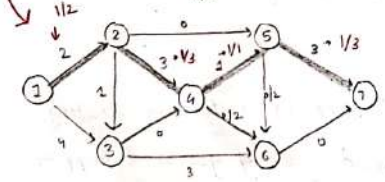


Source : 1

Sink : 7







Bottleneck cap	Augmenting path
5	MIN (1 $\xrightarrow{7}$ 2 $\xrightarrow{5}$ 5 $\xrightarrow{10}$ 7)
2	MIN (1 $\xrightarrow{10}$ 3 $\xrightarrow{2}$ 4 $\xrightarrow{3}$ 5 $\xrightarrow{5}$ 7)
4	MIN (1 $\xrightarrow{8}$ 3 $\xrightarrow{7}$ 6 $\xrightarrow{4}$ 7)
1	MIN (1 $\xrightarrow{2}$ 2 $\xrightarrow{3}$ 4 $\xrightarrow{1}$ 5 $\xrightarrow{3}$ 7)
12	→ Max^m Flow

GRAPH: adjacency matrix code :

```
class graph {
    private: bool ** adjmatrix;
             int numvtx;
    public:  graph (int numvtx) {
                this -> numvtx = numvtx;
                adj matrix = new bool *adjmatrix [numvtx];
                for (int i=0; i<numvtx; i++) {
                        adjmatrix [i] = new bool adjmatrix[numvtx
                        for (int j=0; j<numvtx; j++){
                            adjmatrix [i][j] = False;
                        }
                }
    void addedge (int i, int j) {
                adjmatrix [i][j] = True;
                adj matrix [j][i] = ~~False~~ True;
    }
    void delete_edge (int i, int j){
                adjmatrix [i][j] = False;
                adjmatrix [j][i] = False;
    }
    void display () {
            for (int i=0; i<numvtx; i++) {
                    for (int j=0; j<numvtx; j++){
                        cout << adjmatrix [i][j];
            }       }
    }
```

Scanned with OKEN Scanner

```
~graph(){
        for (int i=0; i<numvtx; i++){
                delete[].adjmatrix[i];
                }
        delete adjmatrix;
    }
};

int main(){

    graph g(7);
    g.addedge(0,2);
              (0,3);
              (0,4);
              (2,4);
              (1,5);
              (2,4) → (1,6)
              (3,5);
              (4,6);
              (5,6);
    g.display();
    delete_edge(6,3);
    delete_edge(5,1);
    g.display();
}
```

GRAPH: adjacency list code.

→ we can consider adjacency list as vector of vectors.

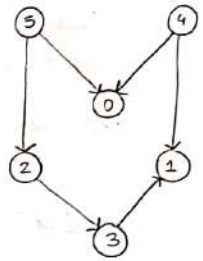
```
class graph {
    private: int numvtx;
             vector<vector<int>> adjlist;
    public: graph(int x){
                numvtx = x;
                adjlist.resize(numvtx);
            }
            void addedge(int src, int dest){
                adjlist[src].push_back(dest);
                adjlist[dest].push_back(src);
            }
            void deleteedge(int src, int dest){
                int destIndex = -1;
                for (int i=0; i<adjlist[src].size(); i++){
                    if (adjlist[src][i] == dest){
                        destIndex = i;
                        break;
                    }
                }
                if (destIndex != -1){
                    adjlist[src].erase(adjlist[src].begin() + destIndex);
                }
            bool isconnected(int src, int dest){
                for (int i=0; i<adjlist.[src].size(); i++){
                    if (adjlist[size][i] == dest){
                        return true;
                    }
                }
                return false;
            }
```

Scanned with OKEN Scanner

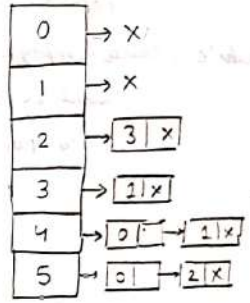
```
void print() {
    for (int i=0; i<< numVtx; i++){
        cout << i << ":";
        for (int j=0; j<< adjlist[i].size(); i++){
            cout << adjlist[i][j]<< " ";
        }
        cout << endl;
    }
}
};

int main() {
    graph g(5);
    g.addedge (0,1);
              (0,4);
              (1,2);
              (1,3);
              (1,4);
              (2,3);
              (3,4);
    g.print();
    g.deleteedge (1,4);
    g.print();
    cout << g.isconnected (0,1) << endl;
    cout << g.isconnected (0,2) << endl;
    return 0;
}
```

Code: Topological Sorting




```
class graph {
    int v;
    list <v> *adj;
    Toposutil (int v, bool visited[], stack<int>& stack);
    public:
        graph (int v);
        void adjedge (int v, int w);
        void toposort();
};
graph :: graph (int v) {
    this -> v = v;
    adj = new list <int> [v];
}
void graph :: addedge (int v, int w){
    adj[v].push_back (w);
}
Void graph :: topoutil (int v, bool visited[], stack<int>& stack){
    visited [v] = true;
    list<int> :: iterator i; -> behaves like a pointer
    for (i= adj[v].begin(); i!=adj[v].end(); i++)
        if (!visited [*i])
            topoutil (*i, visited, stack);
        Stack push (v);
}
```



0	1	2	3	4	5
F	F	F	F	F	F

Scanned with OKEN Scanner

```
void graph :: toposort () {
    Stack <int > Stack ;
    bool * visited = new bool [v];

    for (int i=0; i<v; i++)
            visited [i] = false ;

    for (int i=0; i<v; i++)
            if (visited[i] == false)
            topoutil (i, visited , stack);
    while (stack.empty () == false) {
            cout << stack. top();
            Stack . pop();
    }
}

int main () {
    int n, v1, v2;
    cin >> n ;
    graph g(n)
```

5

SET Operations

① Intersection ② Union ③ Equality ④ Set Differences

① Struct node * get intersection (node* head1, node* head2) {

```
    unordered_map <int, bool >m ;
    Struct node *result = NULL;
    node *p = head1 ;
    while (p! = NULL) {
            m[p→data] = true;
            p = p→ next;
    }
    p = head2;
    while ( p! = NULL) {
            if (m[p→ data] == true) {
                    push (& result, p→data)
                    }
            p = p→ next;
    }
    return result;
}
```

② Struct node * get_union (node * head1 , node * head 2){

```
    Set <int > union_list ;
    node *p= head 1 ;
    while ( p! = NULL) {
            union_list . insert (p→ date) ;
            p = p→ next ;
    }
    p = head2;
    while (p! = NULL) {
            union_list . insert (p→ data) ;
            p=p→ next;
    }
```

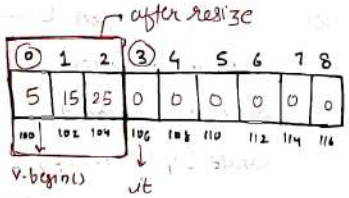
Scanned with OKEN Scanner

```
node * result = NULL;
for ( auto i : union_list ) {
        push (&result, i);
}
return result;
}
```

③ Set equality

```
int main() {
    set <int> m1, m2, m3;
    int i;
    for (i=0; i<3; i++) {
        m1.insert(i);
        m2.insert(i*i);
        m3.insert(i);
    }
    if(m1 == m3)
        cout << " Set m1 and M3 are equal ";
    else
        cout << " Not equal ";
}
```

④ Set difference




```
int main() {
    int first [ ] = {5,10,15,20,25};
    int second [ ] = { 50, 40, 30, 20, 10};
    vector <int> v(10);
    vector <int>:: iterator it;
    sort (first, first+5);
    sort (second, second+5);
    it = set_difference (first, first+5, second, second+5, v.begin());
    v.resize (it - v.begin());
    cout << " The diff "<< (v.size()) << " elements : \n" <<
    for (it = v.begin(); it != v.end(); it++)
```

S1 - S2 = 0 - 3 → -3

→ Representation of sets

→ There are 4 major ways to represent sets.

(i) Linked List [single LL]

(ii) Trees [BST, AVL, Red-black, Splay and B-tree]

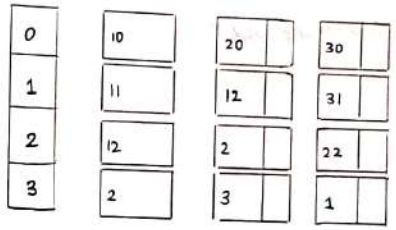
(iii) Hashing

(iv) Bit-vectors

Hashing representation : sets can be efficiently represented using hashing in C++.

→ The SPL provides the unordered set class which is a set implementation based on hashing.

S1 : h(K) = K%10



Scanned with OKEN Scanner

Key advantages :

→ That it provides near constant time avg. complexity for insertion deletion and searching operation.

→ It uses a hash table to internally store the elements making it highly efficient for managing sets.

Eg :

```
#include<unordered_set>
int main() {
  unordered_set<int> myset;
  myset.insert(5);
  myset.insert(3);
  myset.insert(7);
  int valuetofind = 3;
  if (myset.find(valuetofind)! = myset.end()) {
        cout << valuetofind << "is in the set";
        }
  else {
        cout << "Not present";
        }
}
```

Bit Vectors / Bit Set / Bit Array

→ It's a DS that represents a collection of binary values compacted, each element in a bit-vector corresponds to a single bit and these bits collectively represent a sequence of binary values.

→ The visited array in graph is an example of bit vector or bit map.

→ This implementation is efficient in terms of memory usage and provides constant time complexity for insertion, removal and membership checking operation, however it is limited by the range of elements that can be represented in the bit vector.

Q) Compute the sets of no. b/w 0 – 15 that are both prime and odd.

Bit set	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
prime	0	0	1	1	0	1	0	1	0	0	0	1	0	1	0	0
odd	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
prime & odd	0	0	0	1	0	1	0	1	0	0	0	1	0	1	0	0

3 5 7 11 13

$$\left\{ \begin{array}{ll} S1 \cup S2 = S1 \,||\, S2 & S1 \text{ AND } S2 = S1 \,\&\&\, S2 \\ S1 \cap S2 = S1 \,\&\&\, S2 & S1 - S2 = S1 \cap \sim S2 \end{array} \right\}$$

Scanned with OKEN Scanner

Q) BFS is started on a BT beginning from root vertex there is a vertex (t) at a distance (4) from root. If (t) is the (nth) vertex in BFS traversal then find the max^m and min^m possible value of (n). (4, 15)

Q) Consider the hash table with (9 slots). The hash f^n ($h(k) = k \% 9$) the collision are resolved by chaining, the following (9 key) are inserted in order:

[5, 28, 19, 15, 20, 33, 12, 17, 10]

Then find the max^m, min^m & avg. chain length in the hash table.

(a) (3, 0, 1) (b) (4, 0, 1)

(b) (3, 3, 3) (d) (3, 0, 2)

Q) Kanak has of a manager his job is to look at overal pic, he has already noted that (82% of orders) are for (17% of products). He would like to periodically peek at a list ordered by product no. of recently purchased item give his choice for a DS which will lead to be most efficient tree to handle the task and explain his reason for the chosen DS also write the algo for his choice.

Q) How many simple distint graphs can be possible with upto 4 nodes.

for no. of node	1	2	3	4
no. of edges :	0	1	3	6

no. of graphs : 2^0 2^1 2^3 2^6

$1 + 2 + 8 + 64$

$\rightarrow (75)$

Q) Design a hash f^n for the scheme so that it does not suffer from both primary & sec. clustering in add^n, the no. of probing sequences derived from it has to be no less than O (m3)

Scanned with OKEN Scanner

ADSA Pre-Uni Sample paper :

Q1) (i) use bit vector to computes sets of both prime & odd b/w 0-15

bit vector :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
prime bit :	0	0	1	1	0	1	0	1	0	0	0	1	0	1	0	0
odd bit :	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Prime & odd prime && odd :	0	0	0	$\boxed{1}$	0	$\boxed{1}$	0	$\boxed{1}$	0	0	0	$\boxed{1}$	0	$\boxed{1}$	0	0

Set of both prime & odd : (3, 5, 7, 11, 13)

(ii) possible distinct simple graphs upto 4 nodes.

nodes :	1	2	3	4	
possible max^m edges :	0	1	3	6	$\rightarrow \frac{n(n-1)}{2}$

possible max^m simple graphs : $1 + 2 + 8 + 64 = \boxed{75}$

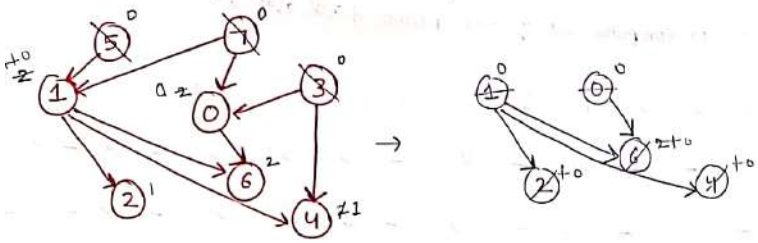
(iii) Ford Fulkerson algo

fordfulkerson (graph, source, sink)

1) Initialize flow = 0.
2) while (there exists an augmented path (P) in the residual graph)
   [2.1] Find the bottleneck capacity.
   [2.2] Augment the path P in residual graph using bottle neck capacity
   [2.3] Update the residual graph.
3) Return the maximum flow.

Scanned with OKEN Scanner

(iv) Kahn's algo for topological ordering :

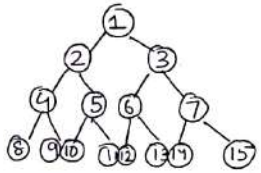


5	7	3

5	7	3	0	1	2	6	4

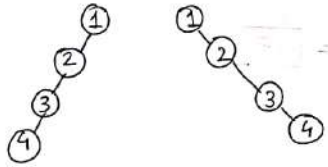
(v) distance : 4 level wise

max: max distance is when there is a complete binary tree



→ $n = 15$ max

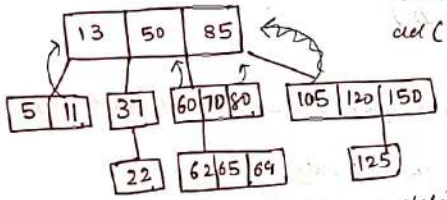
min: min n is when it's a skewed ~~gra~~ tree



→ $n_{min} = 4$

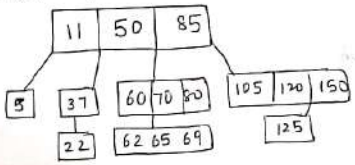
order 4: max child = 4 min child = 2
max key = 3 , min key = 1

(vi)

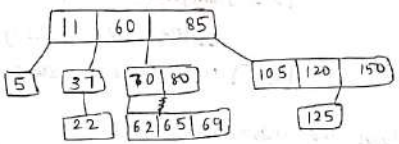


del (13,50,85)

delete (13): replace inorder predecessor



delete (50): replace using inorder successor



delete (85): replace using inorder ~~successor~~ predecessor

(vii) 10 buckets , linear probing , $H(k) = k \cdot 1 \cdot 10$

keys = [43, 165, 62, 123, 142] what is loc. of 142

linear probing: $H(k,i) = [H(k) + i] \% m$ → size

$H(k,i)$ ↓ probe no. ; $H(k)$ ↓ hash fn ; i ↓ index (0-n)

$H(43,0) = (43 \cdot 1 \cdot 10 + 0) \cdot 1 \cdot 10 \to 3$

$H(165,0) = (165 \cdot 1 \cdot 10 + 0) \cdot 110 \to 5$

$H(62,0) = (62 \cdot 1 \cdot 0 + 0) \cdot 1 \cdot 10 \to 2$

$H(123,0) = (123 \cdot 1 \cdot 10 + 0) \cdot 1 \cdot 10 \to 3$ (collission)

↳ $H(123,1) = 4$

$H(142,0) = (142 \cdot 1 \cdot 0 + 0) \cdot 1 \cdot 0 \to 2$ (collission)

↳ $H(142,1) = 3$ (collission)

↳ $H(142,2) = 4$

↳ $H(142,3) = 5$

↳ $H(142,4) = 6$

→ 142 is at 6th index | 7th posn

Key	Location	Probe
43	3	1
165	5	1
62	2	1
123	[3] 4	2
142	[3 4 5] 6	5

0	
1	
2	62
3	43
4	123
5	165
6	142
7	
8	
9	

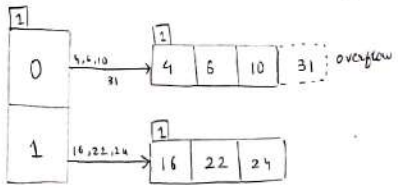
Q2) a) hash fn : X LSB's ∧ X MSB's (Extendible hashing) BS = 3

16 : 10000 22 : 10110 31 : 11111 20 : 10100

4 : 00100 24 : 11000 7 : 00111 26 : 11010

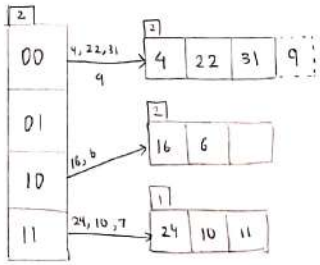
6 : 00110 10 : 01010 9 : 01001

Pass 1 :



16 : 1 ^ 0 = 1 10 : 0
4 : 0 31 : 0
6 : 0
22 : 1
24 : 1

Pass 2 :



16 : 10 00 → 10
4 : 00 00 00
6 : 00 10 10
22 : 10 10 00
24 : 11 00 11
10 : 01 10 11
31 : 11 11 00
7 : 00 11 11
9 : 01 01 00
20 : 10 00 10
26 : 11 10 01

Scanned with OKEN Scanner